Performance evaluation and analysis of Barrelfish using Simics

MATEUSZ OLCZAK

KTH Information and Communication Technology

Bachelor of Science Thesis
Stockholm, Sweden 2013

TRITA-ICT-EX-2013:161

ROYAL INSTITUTE
OF TECHNOLOGY

KTH Royal Institute of technology

Performance evaluation and analysis of Barrelfish using Simics

Bachelor thesis at KTH

Author: Mateusz Olczak

Examinator: Mats Brorsson

Abstract

Personal computing hardware is becoming ever more complex with more cores being added. It is moving from being a multi-core to a many-core system. In the next ten years we are expected to see hundreds of cores on one single chip. It is also very likely we will see more specialized hardware in coexistence with general purpose processing units. The cache-coherent shared-memory operating systems of today do not scale well on the hardware of tomorrow. As the number of cores grows, so does the complexity of the interconnects. The hardware cache-coherence protocols shared-memory operating systems of today rely upon subsequently become increasingly expensive with a greater overhead. As a result, it is entirely possible that operating system of tomorrow will have to handle non-coherent memory. The expected increase in hardware diversity and issues such as cache coherency on hundred core systems poses new challenges for operating system designers. Barrelfish is an research operating system with the purpose of exploring operating-system design of the future. It is an multi-kernel operating system for multi-core systems and utilizes message passing as a way of communication between kernels. Barrelfish assumes no shared-memory, however it does not explicitly forbid shared-memory.

The purpose of this thesis is to performance evaluate Barrelfish using Simics with a modeled approximation of an existing cache structure from a modern processor. A Wool ported version of the Barcelona OpenMP Task suite was used for the purpose of work load simulation, and a comparison between Linux and Barrelfish has been made.

Sammanfattning

Person datorhårdvara bli allt mer komplex med allt fler kärnor. Hårdvaran går från att vara fåkärnig till mångkärnig. Under de kommande tio åren förväntas vi se hundratals kärnor på ett enda chip. Det är också mycket troligt att vi kommer att få se mer specialiserad hårdvara i samexistens med många generella processor kärnor. Dagens cache-koherenta delade minnes-operativsystem skalar dåligt på den hårdvara vi förväntas se framtiden. Då antalet kärnor ökar, ökar även komplexiteten i cache infrastrukturen och den cache-koherens som dagens delade minnes-operativsystem förlitar sig på blir allt dyrare att implementera med allt högre kostnad i form av prestanda. Som ett resultat, är det fullt möjligt att operativsystem i framtiden kommer vara tvungna att hantera icke-koherent minne. Detta tillsammans med förväntad ökning av mångfald av hårdvara, och problem som cache-koherens på system med hundratalet processor kärnor innebär nya utmaningar för operativsystems designers. Barrelfish är ett multikärnigt operativsystem som utnyttjar så kallad message-passing för att kommunicera mellan kärnor. Det utgår inte från att delat minne finns tillgängligt mellan kärnor samtidigt som det inte förbjuder delat-minne.

Syftet med denna avhandling är att utvärdera prestanda hos Barrelfish med hjälp av hårdvarusimulatorn Simics. Tillsammans med Simics har en modellerad approximation av en befintlig cache struktur från en modern processor använts för simuleringar. En Wool porterad version av Barcelona OpenMP Task Suit har använts i syfte att simulera arbetsbelastning. En jämförelse mellan Linux och Barrelfish har gjorts.

Contents

1 Introduction . . . 1
1.1 Background . . . 1
1.2 Problem statement . . . 1
1.3 Purpose . . . 2
1.4 Success Criteria . . . 2
1.5 Limitations . . . 2
1.6 Method . . . 2
2 Background . . . 3
2.1 More cores . . . 3
2.2 Caches . . . 3
2.3 Cache-coherency . . . 4
Snooping . . . 4
Directory-based coherence . . . 4
Read snarfing . . . 4
2.4 The cache-coherent shared-memory model . . . 5
2.5 The message passing model . . . 5
2.6 The issues with and argument for the message passing multi-kernel OS 5
2.7 Barrelfish . . . 6
3 Realization of the Testbed, Barrelfish I/O and Simulations . . . 7
3.1 Configuring Simics using g-cache . . . 7
3.2 Implementing I/O support for Barrelfish under Simics . . . 7
3.3 Simulation setup . . . 8
3.3.1 Simulation limitations . . . 9
4 Simulation Results . . . 10
4.1 Stress. . . . 10
4.2 Fib. . . . 12
4.3 Sort. . . . 14
4.4 FFT. . . . 16
5 Analysis of results . . . 18
5.1 Stress . . . 18
5.2 Fib . . . 18
5.3 Sort . . . 18
5.4 FFT . . . 18
5.5 Discussion . . . 19
5.6 Conclusion . . . 20
6 Summary . . . 21
A Simics benchmarking script . . . 24
B A Wool application in C pseudo code. . . . 25
C Summary of Barrelfish simulations using time slice quantum of 200 ms. 26
C.0.1 Stress on Barrelifsh, time slice quantum of 200 ms. . . . 26
C.0.2 Fib on Barrelfish, time slice quantum of 200 ms. . . . 26
C.0.3 FFT on Barrelfish, time slice quantum of 200 ms. . . . 26

1 Introduction

This section is dedicated to introducing the background of the problem statement and purpose of this thesis paper. It also states the success criteria and the limitations of this work.

1.1 Background

One observed trend in personal computing (PC) hardware is towards multi-core systems with an increasing amount of cores. The next decade will afford us 100's to 1,000's of cores on a single piece of silicon [1, 2, 3]. This means that commodity hardware will become highly more parallel, and mainstream applications will increasingly exploit this hardware [4].

To date, the shared memory model has been the best fit for general purpose computing hardware such as the PC. This both in terms of performance and good engineering, however the trend is shifting. As the number of cores grows, so does the complexity of the interconnects. Hardware cache-coherence protocols will become increasingly expensive to implement and maintain [1, 5].

Cache-coherence requires communication between cores to maintain a consistent state between a set of caches. Traversing a shared data structure in an modern cache-coherent system is equivalent to a series of synchronous remote procedure calls (RPCs) to fetch remote cache lines. For complex data structures, this may result in lots of round trips, whose performance is limited by the latency and bandwidth of the interconnect [6]. With an increasing amount of cores the strain on interconnects grow in a cache-coherent shared-memory (CCSM) system and in turn translates into greater overhead. Some argue, cache-coherency schemes of today do not scale well for the future systems beyond 16 cores [1, 7, 8, 9, 10]. As a result, it is a distinct possibility that future operating systems will have to handle non-coherent memory [1, 2, 5, 7, 11], or will be able to realize substantial performance gains by bypassing the cache-coherence protocol [1, 12].

One suggested solution to reduce the interconnect utilization the shared-memory imposes in multi-core systems or to do away with the need of cache-coherent memory all together is to redesign the operating system and introduce message passing between kernel cores and processes [5, 6]. There are several research operating systems exploring this path, one such initiative is the Barrelfish operating system [7].

1.2 Problem statement

An important factor for operating system (OS) performance is cache usage characteristics. Especially so for shared memory OS-es utilizing many cores, but also for message passing operating systems such as Barrelfish which may be forced to utilize the caches for passing RPC calls on CCSM systems. Cache usage characteristics for the OS and applications may vary greatly between cache designs, and cache designs may even vary greatly between processors in the same processor family. It is thus beneficial to have knowledge about how various cache designs impact the general performance of the OS and applications running on a message passing operating system such as Barrelfish. Analyzing cache usage characteristics for operating systems and applications is common place, but little research has been done regarding the performance of Barrelfish.

1.3 Purpose

For lab testing purposes Simics hardware Simulator is used. Simics will have to be configured as a testbed with various cache hierarchies in which realistic performance benchmarking can be done using the modeled caches. A second goal is to ensure that Barrelfish can communicate with the outside world from inside the testbed with the main purpose of transferring input and output data to and from the environment. The final purpose is to to run Barrelfish and Linux in Simics with the modeled configurations and to compare and investigate which cache structures best fit the message passing paradigm and a message passing based operating system.

1.4 Success Criteria

Ensure network and hard disk I/O in Barrelfish under Simics. Model various caches hierarchies in Simics and analyze of the performance and efficiency of Barrelfish on different such configurations.

1.5 Limitations

Due to time concerns regarding lengthy simulation run times only one cache structure will be analyzed using Simics. The cache is modeled using the Simics g-cache model. For simulation load purposes a subset of Barcelona OpenMP Task Suite[13] (BOTS) applications is used. Since Barrelfish does not have support for OpenMP, all BOTS applications have been ported to Wool[14].

1.6 Method

As the purpose of this thesis work is to investigate cache usage characteristics of Barrelfish using Simics and to ensure network and hard-disk drive I/O for Barrelfish under Simics, an understanding of the workings and limitations of Simics, its g-cache model and supported network interface cards and hard-disk controllers and how they are configured had to be acquired, also modern cache structures had to be researched.

To get a better understanding of the benefits and shortcomings of message passing, shared memory and cache coherency, research was done by reading various scientific papers and literature.

For the purpose of implementing network interface card support a rewrite of the Barrelfish Intel Pro/e1000 driver had to be done as the only commonly supported network interface card by Barrelfish and Simics did not work with the existing Barrelfish driver. This required reading up on the hardware specification for the Intel Pro/e1000 network interface card family, familiarizing one self with the Barrelfish source code, and cross referencing against existing e1000 drivers. In particular Linux and the iPXE - Open Source Boot Firmware source code was looked at but also other open source projects which include a Intel Pro/e1000 network interface card driver.

To get a better understanding of Barrelfish and its source code, several small programs were written for Barrelfish such as a hardware listing utility and a new shell. This proved very useful while analyzing benchmarking results for Barrelfish.

Simics and Simics g-cache was configured in order to perform realistic benchmarking for Linux and Barrelfish using a Wool ported version of the BOTS suite.

2 Background

The motivation for this research is based on the prediction of increased core count and hardware diversity and the problems inherent with increased core count on shared memory architectures. In this section we briefly discuss some of the theoretical background to caches and shared memory systems and the motivation behind research projects such as Barrelfish.

2.1 More cores

In 2001 the capacity was to integrate 1 billion transistors on a 300mm^2 silicon die and the current trend shows us that by 2015 we will have 100 billion transistors on a 300mm^2. Of late, the evolutionary trend has been to invest die real estate in a few large complex processor cores with large caches [11]. Added complexity to the cores will continue [15] however it is very unlikely that it will be the only means of performance gain, trend seem to be shifting. Lately the PC market has seen an increased core count and outside the PC market, processors such as the 16 core SPARC T5, 64 core TilePro, 48 core Intel Single-Chip Cloud Computer (SCC) and the 80 core Intel Polaris research chip.

One argument behind increased core count is evidence suggesting it is not feasible to invest die real estate into ever more complex cores. Performance increase by micro-architecture alone is governed by Pollack's Rule, which states that performance increase is roughly proportional to square root of increased complexity. In other words, if you double the logic in a processor core, it will only deliver 40% more performance. All leading processors in the past have also shown, if you double the logic in a processor core, it only delivers 40% more performance. Thus two smaller processor cores can potentially provide 70 - 80% more performance compared to 40% from a large monolithic core [11]. It is thus more profitable in terms of performance to add more cores, rather than expanding the existing core. Other benefits with increased core count are: each individual core may be turned on and off or frequency scaled to save power, it is easier to load balance among cores to distribute heat across the die, thus improving reliability and power leakage.

Increased core count does however not without complications. New issues are introduced, especially for shared memory systems in terms of cache coherency.

2.2 Caches

As the central-processing-unit (CPU) executes, it fetches data and instructions from random-access-memory (RAM) into registers so it may execute the fetched instructions and processes the data. Accessing the RAM is slow compared to the execution speed of the CPU. An important reason for this is the limited communication bandwidth between the CPU chip and the outside. The implication is that the faster CPU has to stall and wait until the requested instruction or memory becomes available from the slower RAM. This has a significant impact on performance. To make matters worse, the disparity in speed between CPU and RAM has been growing. To date, the trend has been an annual increase in CPU speed at a rate of 50 - 60% while memory speed only improved at a rate of less than 10% [16, 17]. The effect has been dubbed the memory wall [18]. One way to mitigate the problem is to use CPU caches. The CPU cache is a small but very fast memory inside the CPU chip which stores copies of the data from the most frequently used main memory. This works by utilizing the spatial and

temporal locality shown in most programs. Spacial locality gives us that some parts of the program code are executed more often while others are executed less often or not at all. Also the likelihood that data or instruction next to the currently accessed data or instruction will be executed or accessed next is far greater than the likelihood that some randomly picked instruction or data. Temporal locality gives that something which has been used recently might soon be used again. Caches utilize this phenomena by attempting to store these instructions and data. As long as most memory accesses are cached, the average latency of memory will be closer to the cache latency rather than the latency of the main memory. By utilizing caches, CPU performance can thus be greatly increased with reduced stall cycles at a relatively small cost [19, 20].

2.3 Cache-coherency

In multi-core multiprocessor shared-memory system, each core has a separate cache memory, it is thus possible to end up with multiple copies of the same data in a set of caches and one copy in the main memory. When an update occurs on a copy in one of the caches this change has to be propagated throughout all caches sharing that copy in a timely fashion to ensure no core is working on stale data [20]. This is called cache-coherence.

There are three main schemes to maintain cache-coherency: snooping, directory-based coherence and read snarfing.

Snooping In a snooping protocol all caches are accessible via some broadcasting medium, and all caches monitor - or snoop - the medium for changes in cache lines on copies that they also keep. If a change occurs on a copy in some other cache, the cache line is invalidated in the cache of the snooper. Snooping protocols are the most common way to implement cache-coherency but have limits in their scalability due to their broadcasting nature [20]. One way to reduce the snooping traffic is by implementing a snooping filter. The filter may either keep track of what is in the cache or what is not in the cache. In the case that the snoop filter maintains records of cache lines that may be owned by another or several other nodes, the snoop filter may exploit temporal and spatial locality property of some snoop requests by recording blocks that have been snooped recently as and thus invalidated in the cache. A subsequent snoop request for the same block will match in the snoop cache and be filtered [21, 22].

Directory-based coherence Directory-based cache-coherence schemes utilize a common directory to maintain coherence between caches. The directory keeps the state and each processor must ask for permission to load an entry from the primary memory to its cache. When an update occurs the update is either propagated or invalidated in all other caches sharing that copy of the data by the directory. In directory-based coherence the directory most often resides in the main memory [22].

Read snarfing Read snarfing is a snooping protocol based on the heuristic that all invalid blocks will be needed in the future [23]. Cache-coherence is maintained by each cache watching both address and data on a common bus. When a write occurs on data in main memory that the cache is keeping copy of, the cache updates its own copy of the snarfed memory location with the new data. The effectiveness of read snarfing depends highly on the cache size. If the cache size is small, invalid blocks will probably be displaced from the cache, in which case it is not applicable. Also if there

is dynamic change in a block's sharing pattern, read snarfing can hurt performance. Software-controlled updating has been suggested to mitigate this problem [23].

2.4 The cache-coherent shared-memory model

The CCSM model is the dominant model for general purpose computing platforms such as the PC, ARM, IBM, and Oracle (Sun). Communication between processes, sys-calls and communication internally inside monolithic kernel operating systems is done by reading and writing to a shared accessible memory area. Shared memory threads with the same parent may directly access each others data using globals and communication between processes is usually implemented using shared memory inter-process communication (IPC). The ease of use is accomplished by the familiar load and store instructions which read and write to memory, cache-coherency between cores is maintained transparently by the CPU using some cache-coherency protocol. The reason why the CCSM model has proved so popular with in general purpose computing is partly due to legacy reasons and partly because it is generally considered as easy to use and has to date been a good fit both in terms of performance and good engineering. To date the cost of cache-coherency has been worthwhile due to the general benefits.

Considering current trends, parts of the scientific community is arguing that the cost of CCSM is growing. With increasing core count the subsequent complexity and cost of the cache coherence implementation grows, it is thus not given that the CCSM model of today will be the most feasible alternative in the future. However Milo et. al. argue the opposite, that cache-coherency is here to stay, partly due to legacy reasons and partly due to its ease of use and suggest a scalable cache coherence scheme without broadcasting [24].

2.5 The message passing model

Message passing is a communication paradigm where messages are sent by a process or objects to one or more recipients using a message passing interface (MPI). These messages may consist of zero or more bytes and can contain data or a meaning such as signaling or an Remote-Procedure-Call (RPC). With in the field of high-performance computing (HPC) message passing is already an established method of communication. It has been also been shown that message passing can substantially improve performance in multiprocessor systems [1, 25]. One important reason for this is, in distributed systems "chatty" RPCs are reduced by encoding the high-level operation more compactly into one single RPC or code shipping. When communication is expensive (in latency or bandwidth), it is more efficient to send a compact message encoding a complex operation than to access the data remotely[6].

2.6 The issues with and argument for the message passing multi-kernel OS

In the multi-kernel operating system model each core in a multi-core machine has its own kernel. Each kernel and core is then regarded as an independent network node in a distributed system. The system does not assume shared-memory but rather uses message-passing for inter-core and possibly also for interprocess communication [1].

In practice the multi-kernel OS may resemble a micro-kernel architecture with a minimalistic kernel and all functionality such as device drivers and services such as file system in user-space.

The micro-kernel architecture with which the multi-kernel shares a lot of resemblance also uses message-passing as means of IPC. It was much researched in the 1980s as a way to modularize the operating system, however the kernels of the 80s suffered from multiple performance issues and thus turned out to be a disappointment. Today many of the problems have been rectified and message-passing as a means of IPC has been shown to scale better on a large number of cores and outperform shared-memory communication for some classes of problems [1, 25, 26, 27]. As mentioned previously, some argue cache coherence scales poorly. In systems that almost exclusively execute MPI workloads the value of cache coherence in a processor is greatly diminished.

Advocates of the message passing paradigm claim, if hardware support was to be implemented for message passing it would be much cheaper in terms of die real estate than maintaining cache-coherency. It would have a negative impact on some sets of problems, but that impact would mostly be neglect but the overall benefit would be great [27].

Unlike the micro-kernel, the main arguments for the multi-kernel is not modularization of the kernel in itself. The argument is rather that the PC is more and more becoming a networked architecture. The multi-kernel model would better cater to the growing diversity of hardware and cores as well as be able to utilize the benefits that message-passing provides to many problems [1]. The multi-kernel architecture could also enable different instruction-set-architectures (ISA) to coexists heterogeneously on one hardware platform transparent to the user.

2.7 Barrelfish

Barrelfish is a research operating system released by ETH Zürich in Switzerland, with the assistance from Microsoft Research and has the aim to address issues discussed previously, but also to explore the structure of operating system for hardware of the future. It is based on a micro-kernel architecture, where each kernel is offering no more than the necessary support for the underlying architecture. All services and expected functionality exists in user space. All inter-core communication is performed using explicit message passing. Moreover the OS assumes no shared-memory using solely message passing for all inter-core communication. However, it does not forbid the use of shared-memory by applications.

3 Realization of the Testbed, Barrelfish I/O and Simulations

This work had three problem statements: 1) ensure network and HDD I/O, 2) configure Simics with different cache hierarchies, 3) Perform benchmarking simulations using Simics and the modeled cache hierarchies and analyze the results. This chapter is dedicated to the methodology of this work and will explain how the tasks were executed and some of the problems encountered in order to reach the success criteria.

3.1 Configuring Simics using g-cache

For the purpose of cache simulation the default Simics g-cache model was used together with the default MSI snooping protocol. Several cache models were created for comparison, however in the end, due to time concerns regarding exceedingly long simulation runs only the cache model resembling the Intel Nehalem cache was evaluated.

The Intel Nehalem cache is a three level cache design with write-back in all three layers. The 64Kb L1 and 512Kb L2 caches are private for each core, while the 8Mb L3 cache is shared by up to 4 cores. The L1 cache is divided into a 32Kb data- and 32Kb instruction-cache. Since each cache level is write-back a snooper was connected to each cache in the same level in order to maintain cache coherency. The L3 cache was also connected to a memory staller in order to simulate memory access overhead.

During initial trials with Barrelfish and the modeled caches, all cores except the first one only yielded "uncatchable data" statistics. This means the memory access is not permitted to be cached and hence not cached. This issue was tracked down to the Barrelfish kernel not enabling caches for CPU cores. Once the issue was located it was fixed and cache statistics for all CPU cores could be collected.

3.2 Implementing I/O support for Barrelfish under Simics

Another goal of this thesis work was to ensure network and hard disk drive (HDD) support for Barrelfish under Simics. Simics models several different hardware I/O devices such as network cards, HDD controllers, serial ports etc. Which devices are available may vary depending on the Simics license. The available Simics was licensed under Academia License, thus we only had a limited set of hardware to chose from.

For the purpose of implementing network support for Barrelfish, the Intel Pro/e1000 network card family was chosen. Partly to ensure future compatibility as it is a popular and widespread network interface card (NIC) family available on many simulation and hardware platforms, and partly since there already was an existing e1000 NIC driver in Barrelfish. However, it turned out that this driver did not work with the Simics implemented e1000 NIC model. One of the reasons the existing driver did not work was it was not general enough. The driver made assumptions about hardware and thus only supported a limited set of e1000 NICs. This meant the whole hardware initialization part of the driver had to be rewritten from scratch, however the parts of the driver which interface to the network stack could be kept. During the process of writing the new hardware initialization core, cross referencing was done against the hardware documentation and existing open source e1000 drivers to make the driver as general as possible. The existing driver now works under Simics and several other virtualization environments such Qemu, VirtualBox and VMware but has also been tested on real hardware. One bug remains in the EEPROM detection code. For some NICs, including the Simics implemented e1000 NIC the MAC address has to be specified manually.

The HDD controller support proved even more troublesome. Barrelfish implements several SATA controllers but no ATA controller. The WindRiver Simics Academia License for Simics 4.6 only comes with ATA controllers. To implement the ATA controller would have time wise been outside the scope of this thesis work. The department was able to acquire The Intel Simics Academia License for Simics 4.6. This license came with a platform including the ICH10 SATA controller. The existing Barrelfish ICH9 SATA driver was able to detect and use the ICH10 controller.

3.3 Simulation setup

As simulation platform the Simics x86-x58-ich10 motherboard was used. It was then configured with 8 GB of RAM and 8 core-i7-duo processors, totaling 16 cores. As benchmarking OS-es Barrelfish (release2013-01-11) and Linux (Ubuntu 12.04.2 server, kernel 3.5.0-28-generic x86_64) was used.

For benchmarking purposes a subset of wool ported applications of the Barcelona OpenMP Task Suite (BOTS) was used. BOTS is originally a suite of applications written in OpenMP, designed to stress various aspects of multiprocess environments. Wool is a task centric library to spawn and manage tasks but without the need of compiler specific support, hence all OpenMP annotations were removed. All applications in the BOTS suite, both Linux and Barrelfish were compiled with the same gcc compiler binary (gcc version 4.7.3) using the same optimization flags "*-O2 -m64*".

All simulations were performed in an identical manner. First the OS-es were booted and a checkpoint was created. Each simulation run was then performed by running a script which loaded the checkpoint, started the simulation and attached the g-cache. When the program reached the start of the workload, the caches were warmed the with 1 000 000 instructions per core, then the cache statistics were reset and Simics cpu_switch_time was set to 1 where after the simulations were resumed.

At the end of each workload simulation before task tear down, the simulation was stopped and cache statistics were collected as well as statistics provided by the Simics ptime command. Thread spawning, task creation and tear down were thus excluded using breakpoints by the script. Since Barrelfish has no thread migration, thus always has its threads tided to the core they wore spawned on, Linux to was run with tied threads using pthread_affinity. For Linux the threads were tided before the work started. In during the simulations no operations I/O were performed, and since the environment simulation introduces no or very little randomness, consecutive runs always yielded exactly the same results. Hence all simulations were only run once.

Tab. 3.1: Benchmark application and input.

Application	Input	Tied threads
stress[1]	10000 5 15	yes
fib	42	yes
sort	33554432	yes
FFT	16777216	yes

Please refer to Appendix B for Simics benchmarking script and Appendix A for Wool application pseudo code in C.

[1] stress is not a part of the BOTS suite but a wool stress test tool.

3.3.1 Simulation limitations

Only a subset of the wool ported BOTS applications were at hand, and some of the ported applications crashed either under Linux or Barrelfish and thus had to be excluded.

Due to time concerns all applications were only benchmarked at 16 cores using 16 threads with one set of input data for each application on the Nehalem modeled cache hierarchy.

The simulated cache model is only an approximation of its real counterpart.

4 Simulation Results

This section is dedicated to presenting and discussing the Simics simulation benchmarking results.

4.1 Stress.

Tab. 4.1: stress, statistics summary.

	Barrelfish	Linux	difference
Transactions	542385747	8922775	98.35%
IC transactions	375093595	25962403	98.30%
DC transactions	2308497	9367159	98.52%
Runtime cycles	25760216	1286628	95.01%
L1 IC hit rate	100.00%	99.94%	
L2 IC hit rate	31.44%	6.42%	
L3 IC hit rate	43.12%	60.30%	

Tab. 4.2: stress, L1 I/D fetch statistics.

Core	Barrelfish IC fetch	Linux IC fetch	Barrelfish DC fetch	Linux DC fetch
0	5719391	66429	1748990	17502
1	23960286	1286643	6594138	326743
2	24340285	1286643	6698921	326870
3	23278433	1246669	6408540	316597
4	24811979	1209907	6867989	307650
5	25169247	670178	6923785	170429
6	23181661	544864	6408637	138634
7	25011486	24868	6879528	6807
8	24643376	4384	6781147	1200
9	25460289	4084	7001852	1127
10	25612344	4092	7044652	1129
11	25510361	4072	7016806	1125
12	25560288	4080	7029877	1127
13	25233593	4116	6939992	1135
14	23220293	4096	6392433	1131
15	24380283	4132	6709036	1139

Stress, L1 transactions in total.

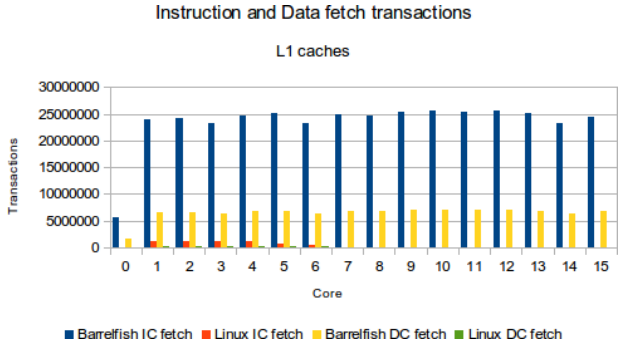

Fig. 4.1

Stress, Cache instruction hit ratios.

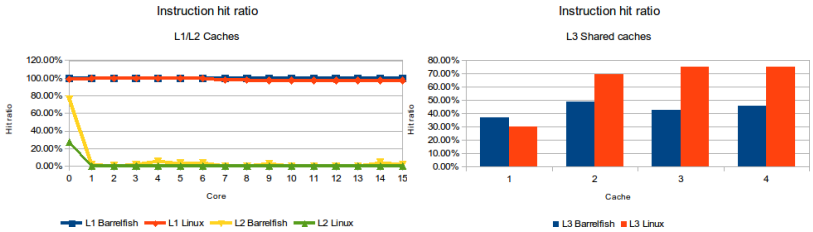

Fig. 4.2

Stress, Cache data read hit ratios.

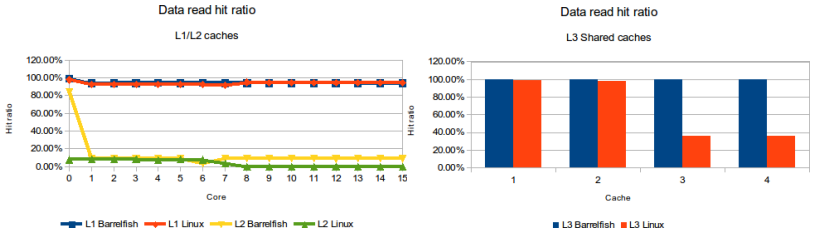

Fig. 4.3

Stress, Cache data write hit ratios.

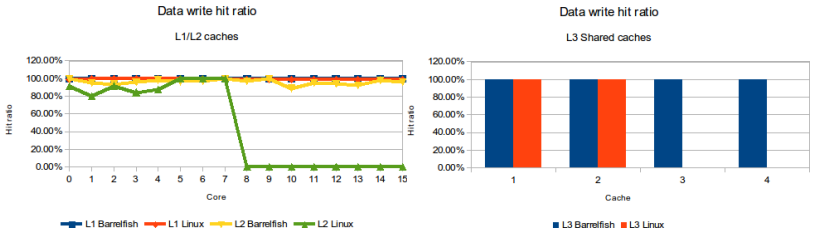

Fig. 4.4

4.2 Fib.

Tab. 4.3: fib, statistics summary.

	Barrelfish	Linux	Difference
Transactions	35823293566	29460660034	17.76%
IC transactions	23786182540	19882973127	16.41%
DC transactions	12029487185	9555669773	20.56%
Runtime cycles	1492715517	1345640578	9.85%
L1 IC hit rate	100.00%	100.00%	
L2 IC hit rate	2.92%	84.31%	
L3 IC hit rate	35.61%	56.15%	

Tab. 4.4: fib, L1 I/D fetch statistics.

Core	Barrelfish IC fetch	Linux IC fetch	Barrelfish DC fetch	Linux DC fetch
0	1469135521	1310006747	414608914	364109538
1	1489022399	1321766870	420470637	366209066
2	1491348361	1321826926	420496560	364935542
3	1490194263	1321847071	418231845	365326635
4	1491575524	1321730157	418786312	367499391
5	1481575645	1328440075	418417724	367351166
6	1489834459	1329340089	419830984	368153692
7	1470866559	1328858146	415329285	367171449
8	1490825299	1328638150	420256322	365589847
9	1490335539	1328853188	418156686	365193548
10	1491635524	1328569289	420088108	366830844
11	1489675547	1328127052	418220109	365439335
12	1490331143	1327947016	418673303	367512556
13	1487815569	1327922889	418250483	365758665
14	1480455662	1327942585	418458852	365708175
15	1491555526	1156877	418491287	322459

Fib, L1 transactions in total.

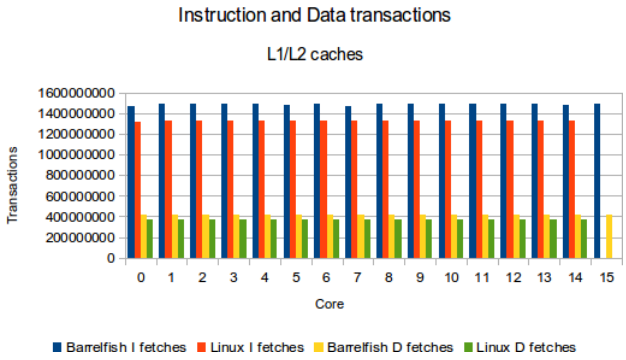

Fig. 4.5

Fib, Cache instruction hit ratios.

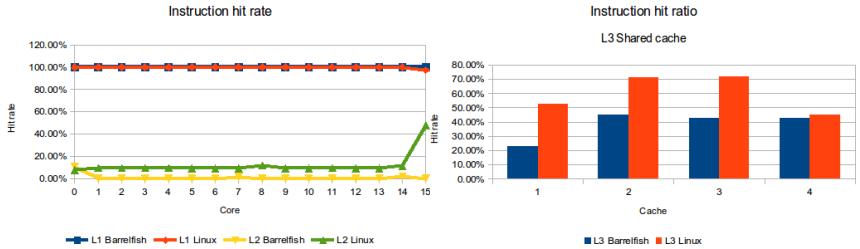

Fig. 4.6

Fib, Cache data read hit ratios.

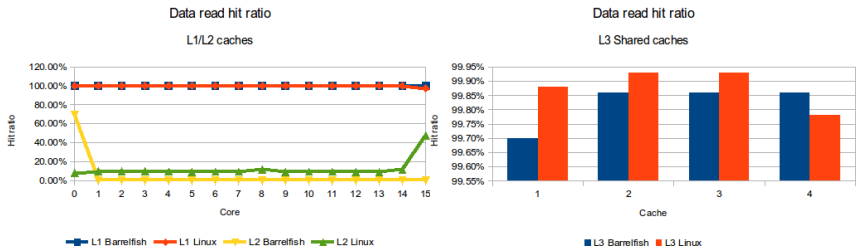

Fig. 4.7

Fib, Cache data write hit ratios.

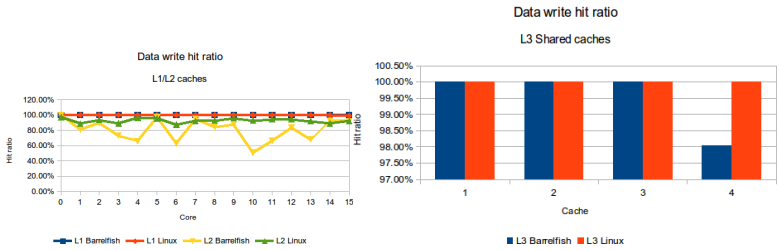

Fig. 4.8

4.3 Sort.

Tab. 4.5: sort, summary.

	Barrelfish	Linux	Difference
Transactions	81412474527	46349368321	43.07%
IC transactions	57282531760	34195355713	40.30%
DC transactions	22013851328	11219290836	49.04%
Runtime cycles	3585381175	2317574019	35.36%
L1 IC hit rate	100.00%	100.00%	
L2 IC hit rate	82.95%	62.20%	
L3 IC hit rate	18.01%	40.28%	

Tab. 4.6: sort, L1 I/D fetch statistics.

Core	Barrelfish IC fetch	Linux IC fetch	Barrelfish DC fetch	Linux DC fetch
0	3540333481	2262657395	858819132	416891115
1	3583161213	2213643307	908217237	495472589
2	3583141213	2276366388	908128036	510997248
3	3583161213	2276545108	908343044	503944621
4	3582142353	2276309148	907777249	505026387
5	3583161213	2290186168	907754068	515245530
6	3582240940	2290022618	907558192	507272340
7	3583181213	2289602436	907820442	512334221
8	3583121213	2289959086	908743495	508293707
9	3583181213	2289517598	907493737	506139209
10	3582813483	2288974169	908730500	507180180
11	3583181213	2287972646	907866736	510400819
12	3582972421	2285665395	910937837	501607824
13	3582677064	2288729829	906681236	510193764
14	3581910428	2287965782	907334727	507265705
15	3582151886	1238640	906810905	346907

Sort, L1 transactions in total.

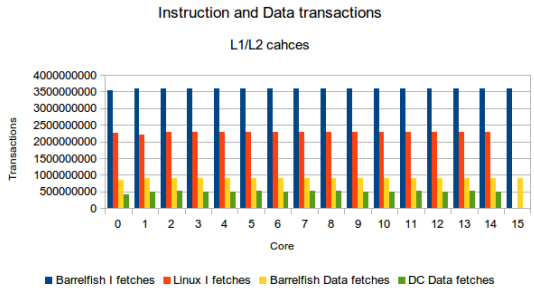

Fig. 4.9

Sort, Cache instruction hit ratios.

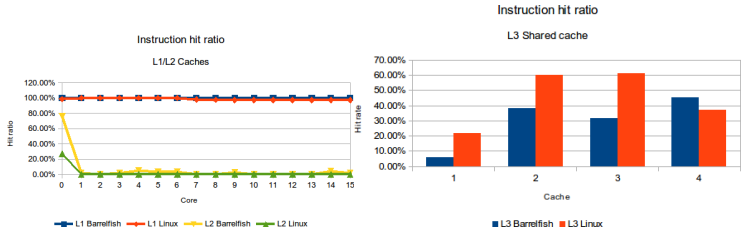

Fig. 4.10

Sort, Cache data read hit ratios.

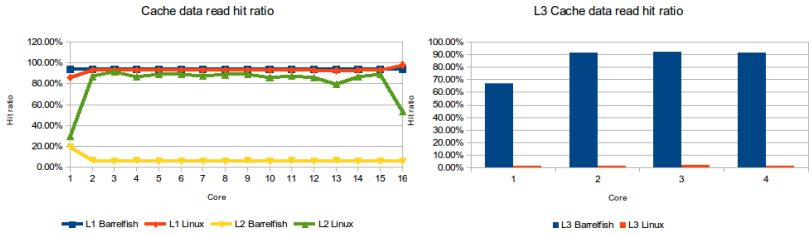

Fig. 4.11

Sort, Cache data write hit ratios.

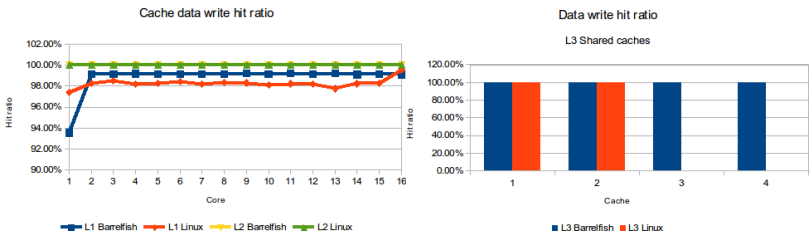

Fig. 4.12

4.4 FFT.

Tab. 4.7: FFT, statistics summary.

	Barrelfish	Linux	difference
Transactions	91144789776	35823141800	60.70%
IC transactions	62456380724	23785858175	61.92%
DC transactions	25894461370	12029351967	53.54%
Runtime cycles	3906722720	1492675513	61.79%
L1 IC hit rate	100.00%	100.00%	
L2 IC hit rate	26.94%	34.40%	
L3 IC hit rate	85.62%	35.87%	

Tab. 4.8: FFT, L1 I/D fetch statistics.

Core	Barrelfish IC fetch	Linux IC fetch	Barrelfish DC fetch	Linux DC fetch
0	3872223886	1469135458	1103361848	414608901
1	3905848361	1489021784	1076796882	420470365
2	3905870051	1491347747	1077071022	420496284
3	3904864936	1490193648	1076739699	418231573
4	3905885761	1491535524	1076704585	418775303
5	3905615008	1481535645	1076850772	418406723
6	3905077846	1489833844	1076245978	419830704
7	3905853496	1470865945	1076989780	415329009
8	3905891026	1490824684	1076729510	420256042
9	3905890116	1490295539	1076320198	418145653
10	3905496886	1491595524	1076221170	420077080
11	3905889076	1489635547	1076504357	418209129
12	3905868556	1490330529	1076362998	418673035
13	3905773790	1487775569	1076616625	418239465
14	3904937653	1480415662	1076460997	418447850
15	3905394276	1491515526	1076848035	418480319

FFT, L1 transactions in total.

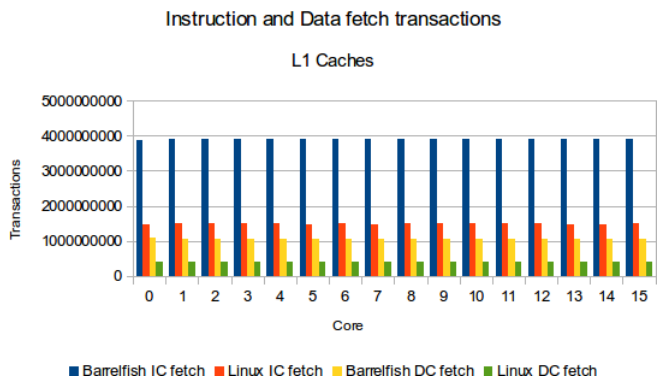

Fig. 4.13

FFT, Cache instruction hit ratios.

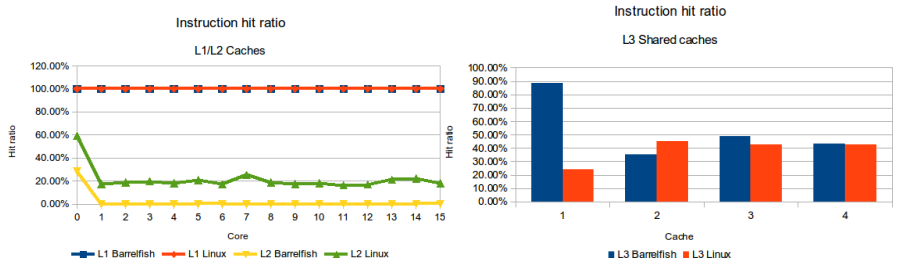

Fig. 4.14

FFT, Cache data read hit ratios.

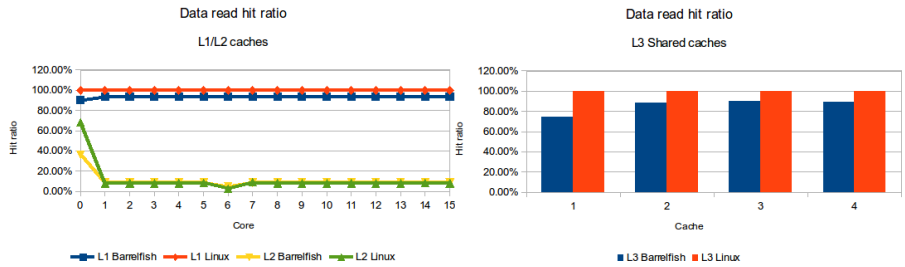

Fig. 4.15

FFT, Cache data write hit ratios.

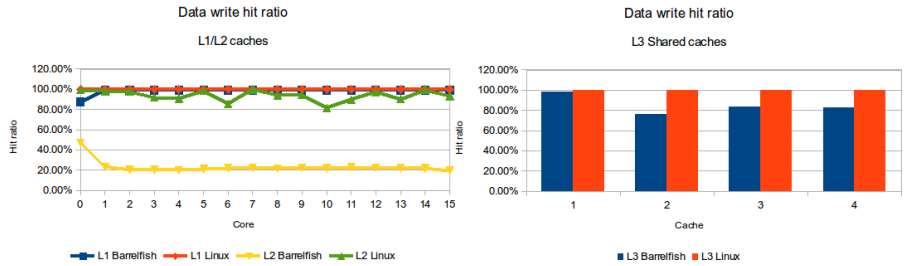

Fig. 4.16

5 Analysis of results

This part of the document is dedicated to analysis of the simulation runs.

5.1 Stress

Stress is the only application not part of the BOTS suite. It has a a ridiculously parallel work load designed to stress test the work queue of Wool by creating a large number of tasks which simulate work by iterating a local variable in a for loop.

Stress has the shortest execution time of all the benchmark applications used in this benchmark. For some reason stress workload turns out to be very unevenly balanced among threads for Linux. By looking at the instruction and data fetches in Table 4.2 we see that the majority of work were done by threads 6 to 7. This is possibly due to small task granularity. Consecutive failed work stealing attempts by a wool thread will result in an increased back-off delay for the next steal attempt. If the threads with work manage to finish their tasks before the back-off delay has Finnish, no work or little work will be stolen by threads with small task queues, thus they will have completed a small amount of tasks.

Barrelfish on the other hand has an even amount of work distributed among all threads how ever, we see in Table 4.1 Barrelfish uses almost double the amount of instruction and data fetches to complete the work, hence it is almost twice as slow as the Linux stress. Note worthy is that both Linux and Barrelfish have similar L1 cache hit ratio, around 100%. This hints that Barrelfish poor performance has nothing to do with cache misses.

5.2 Fib

Fib is the BOTS application dedicated to simulate workload by calculating the Fibonacci numbers. It Table 4.4 we see that work among all threads, Linux and Barrelfish is even. Again Barrelfish uses more instructions to complete the work and is hence slower then the Linux fib, but this time only 10% slower. Both operating systems have around 100% L1 hit rate, the poor L2 instruction hit rate shown in Table 4.4 is neglectable since it is less then 0.001% of the total instruction fetches.

5.3 Sort

Again Linux is much quicker, about 35% for the BOTS sort benchmark, and again it is related to the number of instruction and data fetches rather then cache hit rate.

5.4 FFT

In the BOTS Fast Fourier Transforms benchmark the previous pattern is repeated. Again Barrelfish and Linux have similar hit rates in the caches but Barrelfish is this time 62% slower due to a total of 61% more instruction and data transactions.

5.5 Discussion

On average Linux is overall 43% faster than Barrelfish in the benchmarks, however the performance difference is not due to poor cache hit rates on Barrelfish behalf but tied to the generally higher instruction and data fetch count. I argue that there are three possible reasons for this:

1. Some other interfering application.
2. There is an issue in the Barrelfish kernel.
3. There is an issue in Barrelfish application layer.

Interfering application.

Barrelfish does not have thread migration, and no other applications were spawned on cores 1 to 15. The only software running on those cores is the benchmarking software and the kernel. I hence believe application interference can be excluded.

The kernel.

In order to attempt to find an explanation in the kernel I looked at the Barrelfish kernel source code, but was not able to find any thing that stands out, on the contrary. If work is present the scheduler schedules that work for execution, if no work is available, the kernel sleeps by executing a halt loop while waiting for a timer interrupt. Even if the kernel was sleeping, this behavior should not yield any excessive amount instruction fetches and excludes the data fetches we are seeing.

However I was able to find one general difference between Linux and Barrelfish. The context switch time slice quantum for Barrelfish is by default 80ms and for Linux it is calculated to 200ms our machine. That is a 60% difference and should only have a minor impact on the performance due to the increased context switching done by Barrelfish. To confirm this I recompiled Barrelfish using a time slice quantum of 200ms and ran yet another limited set of experiments to investigate any performance difference.

For stress, the application with biggest performance difference the a speedup of 5 percentage units was achieved due to less context switching as shown in Table C.1, however for fib there was a performance degradation of 0.62 percentage units as shown in Table C.2.

The available version of Simics no longer supported the pstats command. It would have helped us differentiate between time spent in supervisor mode aka. kernel-mode and user-mode. This would have helped us to exclude with certainty any misbehavior of the kernel, however I find it very unlikely the kernel is the source and hence argue that the issue at hand has to be found in the application layer.

The application layer.

As a starting point I looked at Wool and its thread locking. For Linux pthread_mutex_locks are used and under Barrelfish the native thread_mutex_lock is used. Here I was able to find a significant implementation difference. Linux uses a system called Fast user space mutex (Futex). In this system an atomic increment and test operation is performed on a mutex variable in user space. If the result of the operation indicates that there was no previous locking the call to pthread_mutex_lock returns with out ever

switching into kernel context. Only if lock contention is detected, the process will perform the Futex system call and context will switch into kernel mode which puts the calling thread to sleep until the mutex is released.

In Barrelfish the native thread_mutex_lock is a library function and implemented as a spin lock. When lock contention is detected, the thread spins on the lock until it is released and the lock acquired. This means that the thread never yields or sleeps but does a lot of instructions and data fetches to check lock status. This also has the additional side effect of increased congestion on the cache-coherency interconnects.

Yet another telltale sign that his might be the issue is the huge performance difference between fib and stress compared to Linux. My conclusion is that, looking at the stress benchmark instruction count for Linux, what looks like uneven task load for Linux, is in fact the benefit of Futex as the thread yields until the lock becomes available. I also argue the reason why fib on Barrelfish is only about 10% slower then on Linux, as shown in Table 4.3 is that the task granularity varies more and less lock contention occurs, hence less spin locking and less performance impact.

5.6 Conclusion

Barrelfish is a research operating system and work in progress while Linux kernel is a very optimized and mature product. Even in this limited set of benchmarking applications it is evident that Barrelfish suffers from performance issues. Through method of exclusion the issue most likely lies within the thread library due to spin locks.

6 Summary

Of late, the evolutionary trend has been to invest die real estate into a few large complex processor cores with large caches. However the trend has now started to shift towards increased core count. This coupled with predictions of more diversified hardware on one and the same hardware platform will present new issues to operating system designer. This has inspired research projects on operating systems of the future. One such project is Barrelfish.

As a contribution to that research, this thesis work has contributed to the Barrelfish source tree with several patches. The main patch has been the rewrite of the Barrelfish Intel Pro/e1000 network interface card driver. The new driver is more generic and simpler to extend it with support for new cards as well as it has added support for several existing Intel Pro/e1000 network interface cards.

Using Simics coupled with its g-cache model, benchmarking has performed in a realistic manner. Analysis of the results have shown performance issues in Barrelfish. By investigating the Barrelfish source code this thesis work has also been able to give a plausible explanation of their source. Unfortunately due to time constraints we were not able to benchmark all the modeled caches.

References

[1] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, Akhilesh Singhania. The Multikernel: A new OS architecture for scalable multicore systems, In Proceedings of the 22nd ACM Symposium on OS Principles, Big Sky, MT, USA, October 2009.

[2] D. Wentzlaff and A. Agarwal. Factored operating systems (fos): The case for a scalable operating system for multicores. Operating Systems Review, 43(2), Apr. 2009. pp. 76-85

[3] T. G. Mattson, R. Van der Wijngaart, and M. Frumkin. Programming the Intel 80-core network-on-a-chip terascale processor. Proceedings of the 2008 International Conference on Supercomputing. 2008.

[4] Simon Peter, Adrian Schüpbach, Paul Barham, Andrew Baumann, Rebecca Isaacs, Tim Harris, Timothy Roscoe. Design Principles for End-to-End Multicore Schedulers, 2nd Workshop on Hot Topics in Parallelism, Berkeley, CA, USA, June 2010.

[5] Niladrish Chatterjee, Seth H. Pugsley, Josef Spjut, Rajeev Balasubramonian. Optimizing a Multi-Core Processor for Message-Passing Workloads, The Journal of Supercomputing, 2012, Vol.60(1), pp. 141-162

[6] Andrew Baumann, Simon Peter, Adrian Schüpbach, Akhilesh Singhania, Timothy Roscoe, Paul Barham, Rebecca Isaacs. Your computer is already a distributed system. Why isn't your OS?, Proceedings of the 12th Workshop on Hot Topics in Operating Systems, Monte Verità, Switzerland, May 2009.

[7] Barrelfish Operating System, http://wiki.barrelfish.org/

[8] B. Choi, R. Komuravelli, H. Sung, R. Bocchino, S. Adve, and V. Adve. DeNovo: Rethinking Hardware for Disciplined Parallelism. Proceedings of the Second USENIX Workshop on Hot Topics in Parallelism (HotPar), 2010.

[9] J. Howard et al. A 48-Core IA-32 Message-Passing Processor with DVFS in 45nm CMOS. Proceedings of the International Solid-State Circuits Conference, 2010. pp. 108-109.

[10] J. H. Kelm, D. R. Johnson, W. Tuohy, S. S. Lumetta, S. J. Patel. Cohesion: An Adaptive Hybrid Memory Model for Accelerators. IEEE Micro, 31(1), January/February 2011. pp. 42-55.

[11] S. Borkar. Thousand core chips: a technology perspective. Proceedings of the 44th Annual Design Automation Conference. 2007. pp. 746–749.

[12] D. Wentzlaff, P. Griffin, H. Hoffmann, L. Bao, B. Edwards, C. Ramey, M. Mattina, C.-C. Miao, J. F. Brown, III, A. Agarwal. On-chip interconnection architecture of the Tile Processor. IEEE Micro, 2007. pp. 15-31.

[13] Barcelona OpenMP Task Suite (BOTS): http://pm.bsc.es/projects/bots

[14] Wool, http://www.sics.se/∼kff/wool/

[15] Shekhar Y. Borkar, Hans Mulder, Pradeep Dubey, Stephen S. Pawlowski, Kevin C. Kahn, David J. Kuck, Justin R. Rattner, Platform 2015: Intel Processor and Platform Evolution for the Next Decade, 2005.

[16] D.A. Patterson and J.L. Hennessy, Computer Architecture: A Quantitative Approach, 2nd ed., Morgan Kaufmann, 2012.

[17] Vikas Agarwal, M.S. Hrishikesh, Stephen W. Keckler, Doug Burger, Clock Rate versus IPC: The End of the Road for Conventional Microarchitectures, Proceedings of the 27 Annual International Symposium on Computer Architecture, XXX

[18] Win. A. Wulf, Sally A. Mc Kee, Hitting the Memory Wall: Implications of the Obvious, ACM SIGARCH Computer Architecture News, 1995, Vol.23(1), pp.20-24.

[19] Fredrik Lundevall, Cacheminne och adressöversättning, 2003, http://web.it.kth.se/∼flu/papers/cmmu.pdf

[20] David A. Patterson, John L. Hennessy, Computer organization and design, 4th ed., pp 452-454, 534-538

[21] Valentina Salapura, Matthias Blumrich, Alan Gara, Design and Implementation of the Blue Gene/P Snoop Filter, 2008 IEEE 14th International Symposium on High Performance Computer Architecture, Feb. 2008, pp. 5-14.

[22] Samaher Al-Hothali, Safeeullah Soomro, Khurram Tanvir, Ruchi Tuli, Snoopy and Directory Based Cache Coherence Protocols: A Critical Analysis, Journal of Information & Communication Technology Vol. 4, No. 1, 2010.

[23] Aleksandar Milenkovic. Achieving High Performance in Bus-Based Shared-Memory Multiprocessors, IEEE Concurrency, 2000. pp. 36-44

[24] Martin, Milo, M. K., Hill, Mark, D., Sorin, Daniel, J. Why on-chip cache coherence is here to stay. Communications of the ACM, July 2012, Vol.55(7), pp. 78-89

[25] David Kranz , Beng-Hong Lira, Kirk Johnson, John Kubiatowicz, Anant Agarwal. Integrating Message-Passing and Shared-Memory: Early Experience, PPOPP '93 Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming, pp 54-63

[26] Yong Luo. Shared Memory vs. Message Passing: the COMOPS Benchmark Experiment, The Journal of Supercomputing, 13, 1999. pp.283-301

[27] Jochen Liedtke. Improving IPC by kernel design. ACM SIGOPS Operating Systems Review, 1993, Vol.27(5). pp. 175-188

Appendix

A Simics benchmarking script

```
## simics MUST be launched with -stall option
add-directory "%script%"

# Load Simics chekpoint
read-configuration after_boot.ckpt
run-command-file system.simics
#######################
# Initialize statistics
#######################
script-branch{
        $system.serconsole.con.wait-for-string "INIT*"
        stop
        # Start the g-cache
        run-command-file targets/bf/add-gcache.include
        # warm-up the caches
        c 1_000_000
        # reset cache statistics
        foreach $c in ($caches) {
                $c.reset-statistics
        }
        # Collect ptime statistics
        output-file-start -overwrite logs/ptime.log
        ptime -all
        cpu-switch-time 1
        stop
        run
}
#######################
# Collect statistics
#######################
script-branch {
        $system.serconsole.con.wait-for-string "END*"
        stop
        ptime -all
        output-file-stop
        # Collect g-cache statistics
        output-file-start -overwrite logs/gcache.log
        foreach $c in ($caches) {
                $c.statistics
        }
        output-file-stop
        cpu-switch-time 1000
}
#######################
# Stop benchmarking
#######################
script-branch {
        if $system == Barrelfish {
                $system.serconsole.con.wait-for-string "/␣>"
        } else {
                $system.serconsole.con.wait-for-string "mao@ubuntu:~$"
        }
        stop
}
$system.serconsole.con.capture-start console-output.log
$system.serconsole.con.input "stress␣-p␣16␣10000␣5␣15\n" 0
run
```

B A Wool application in C pseudo code.

```c
#define WOOL_INIT()     start_work()
#define WOOL_FINI()     stop_work()

/* Wool worker thread */
void wool_do_work(void *arg)
{
        /* wait for all worker threads to come up */
        wait_for_init_done();
        /* do work */
        ...
#ifdef BARRELFISH
        wool_join();
#endif
}

/* Wool library function to start worker pool. */
void wool_start_work()
{
        /* spawn worker threads */
        for(i = 0; i < n_workers; i++) {
                create_thread(do_work, arg);
        }

        /* wait for all worker threads to come up */
        wait_for_init_done();

        /* Used as Simics script break point
         * to start statistics gathering. */
        print("INIT*");
}

/* Wool library function to join worker threads. */
void wool_stop_work()
{
        print("DONE*");

        /* join worker threads */
        join_threads();

        /* Used as Simics script break point
         * to end statistics gathering. */
        print_worker_statistics();
}

/* Main function of application using wool. */
int main(int argc, char *argv[])
{
        /* Init tasks and start workers */
        WOOL_INIT();

        /* Create tasks using wool and do work.
         * This thread will also be a worker */
        ...

        /* Wait for work to complete,
         * then join worker threads. */
        WOOL_FINI();

        return 0;
}
```

C Summary of Barrelfish simulations using time slice quantum of 200 ms.

C.0.1 Stress on Barrelifsh, time slice quantum of 200 ms.

	Barrelfish	Linux	difference	diff. % units
Transactions	28227143	8922775	96.84%	1.51%
IC transactions	195151048	25962403	96.74%	1.56%
DC transactions	81230526	9367159	97.16%	1.36%
Runtime cycles	13357241	1286628	90.37%	4.64%
L1 IC hit rate	100.00%	99.94%		
L2 IC hit rate	19.61%	6.42%		
L3 IC hit rate	37.50%	60.30%		

Tab. C.1

C.0.2 Fib on Barrelfish, time slice quantum of 200 ms.

	Barrelfish	Linux	difference	diff. % units
Transactions	36043406379	29460660034	18.26%	-0.50%
IC transactions	23937193772	19882973127	16.94%	-0.53%
DC transactions	12098169931	9555669773	21.02%	-0.45%
Runtime cycles	1503027357	1345640578	10.47%	-0.62%
L1 IC hit rate	100.00%	100.00%		
L2 IC hit rate	28.61%	84.31%		
L3 IC hit rate	38.88%	56.15%		

Tab. C.2

C.0.3 FFT on Barrelfish, time slice quantum of 200 ms.

	Barrelfish	Linux	difference	diff. % units
Transactions	91255427117	35823141800	60.74%	-0.07%
IC transactions	62533518509	23785858175	61.96%	-0.05%
DC transactions	25925615627	12029351967	53.60%	-0.06%
Runtime cycles	3911467045	1492675513	61.84%	-0.05%
L1 IC hit rate	100.00%	100.00%		
L2 IC hit rate	29.12%	34.40%		
L3 IC hit rate	88.29%	35.87%		

Tab. C.3

TRITA-ICT-EX-2013:161

www.kth.se